Way back in chapter 2 (it seems like so long ago, before we were friends), you wrote your first view: `credits()`. It is only appropriate that your first test should be for that very same view. The purpose of your test is to exercise the view and validate the results returned from it. Figure 8.2 shows the interleaved execution of a test for `credits()` and the view itself.

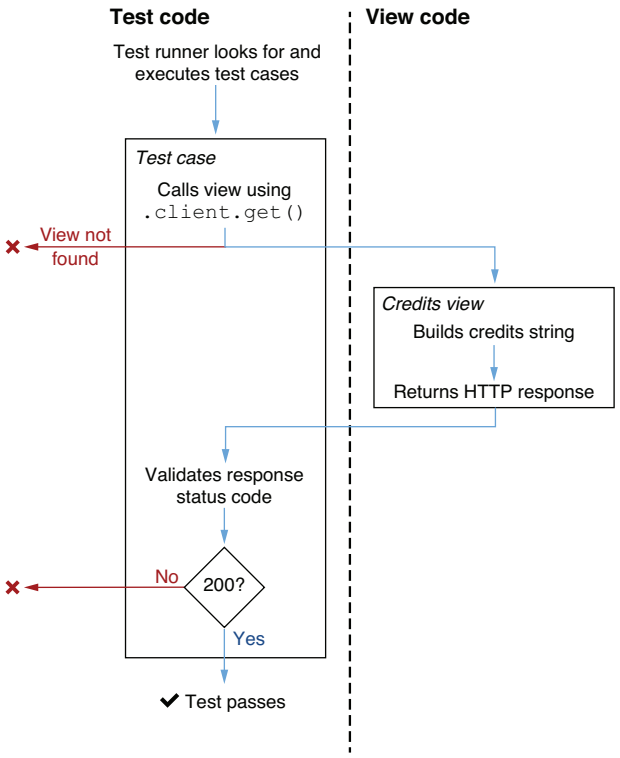


Figure 8.2 Testing the `credits()` view

As the `credits()` view and its corresponding test case are both in the home app, a decent name for the test case is `TestHome`. Inside this class, you write a method called `.test_credits()`, which is the code that tests the view.

Django's version of the `TestCase` class includes an object called `.client` that you use to interact with the website. The `.client` object has both `.get()` and `.post()` methods, emulating a browser interaction. Note that it only emulates it, so the template gets rendered, but no browser is involved. If there is JavaScript on the page, it doesn't get run. Using `.client.get()` is almost like calling a view function directly, except it does it through the URL routing system.

Once you've invoked a view, you want to make sure it works properly. For a simple view, like `credits()`, there isn't much to test besides getting back the right HTTP status code. In this case, the expected value is `200`, the HTTP response code for "all good." To ensure you received this value, you call `.assertEqual()`. This is one of